URL shortener

TLDR

This document outlines the design for a high-performance URL shortening service, prioritizing high availability and low latency to ensure a seamless user experience.

Requirements

Functional Requirements

Functional requirements are the features that the system must have to satisfy the needs of the user.

1. **Core Feature**: Users can submit a long URL and receive a unique short URL.
2. **Redirection**: Visiting the short URL redirects to the original URL.
3. **Customization**: The service supports custom aliases.
4. **Analytics**: Basic tracking of click counts for each shortened URL.

Non-Functional Requirements

Non-functional requirements refer to specifications about how a system operates, rather than what tasks it performs.

1. **High availability** : The system must prioritize availability over strict consistency.
2. **Low latency**: Redirection should be extremely fast, with a target $P99 < 50ms$
3. **Scalability:** Must handle 1 billion reads and 10 million writes daily (100:1 read-write ratio) using horizontal scaling.
4. **Durability**: Once a mapping is created, it must not be lost.
5. **Uniqueness**: Short URLs must be globally unique.

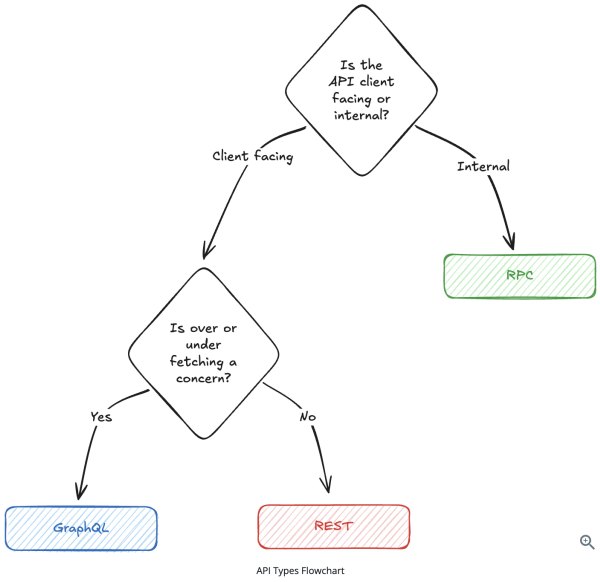
Capacity Estimation

Based on a **100:1 read-write ratio**:

- **Writes (URL Generation)**: 10M/day ≅ **115 QPS** (Peak: 350 QPS).
- **Reads (Redirection)**: 1B/day ≅**11,500 QPS** (Peak: 35,000 QPS).

API Design

RESTful APIs will be used for client-facing communication, since each endpoint returns exactly the expected resource and does not introduce overfetching or underfetching.



Taken from [hellointerview]

Endpoints

Shorten a URL - POST /urls

The endpoint take a long url and return a short one

Request Body

```
{
  "long_url": "https://www.example.com/some/very/long/url",
  "custom_alias": "optional_custom_alias",
  "expiration_date": "optional_expiration_date"
  "User_id" : "user"
}
```

Response Body

```
{
 "short_url": "http://short.ly/abc123"
}
```

Error cases:

- **400 Bad requests** - the long URL is invalid.
- **409 Conflict** - the custom_alias URL already exists.
- **401 Unauthorized** — missing or invalid authentication token.
- **429 Too Many Requests** — per-user or global rate limit exceeded.

Redirect to the Original URL - GET /{short_url}

The endpoint takes the short URL and redirects the user to the original long URL.

Response example

```
HTTP 302 Redirect to the original long URL
```

Response code:

- **302 Found (default):** Temporary redirect. Every request passes through the server, ensuring **click events are captured for analytics**. This is the correct default.
- **301 Moved Permanently:** Browsers cache this response permanently and replay it locally, bypassing the server on repeat visits. This means **no click events are generated.** Only use 301 when the user explicitly opts into a permanent, analytics-free redirect.
- **410 Gone:** The link has expired.
- **404 Not Found:** The link doesn't exist.

Get URL Analytics

GET /analytics/{short_url}

Response Body

```
{
  "short_url": "http://short.ly/abc123",
  "click_count": 120,
  "last_visited": "2026-03-02T12:45:00Z",
 "created_at": "2026-01-01T12:45:00Z"
}
```

DataBase Design

To handle billions of records with high read throughput, a **NoSQL database (e.g., DynamoDB)** is enough..

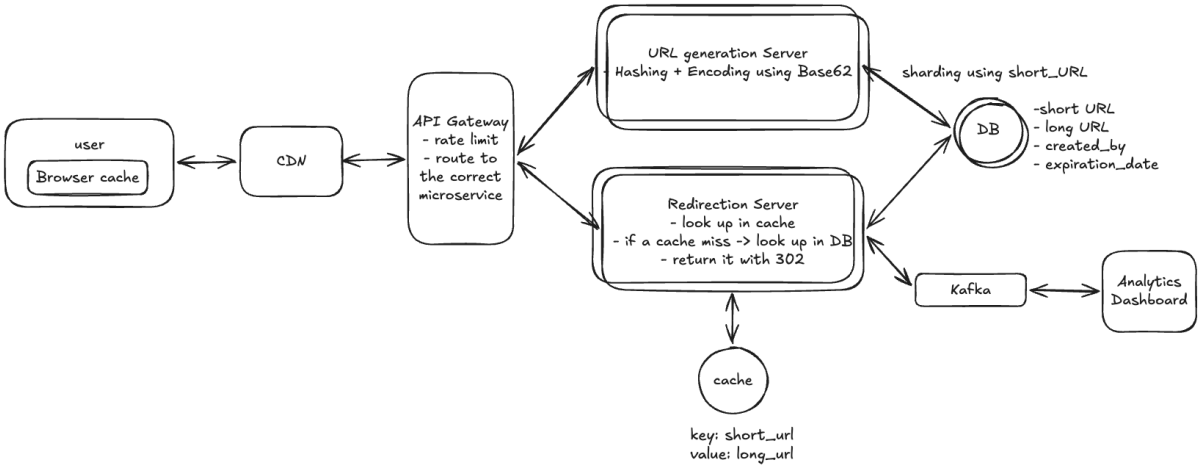
- **Schema**: Simple key-value mapping of {short_url: long_url}.
- **Metadata**: Store created_by, created_at, and expiration_date.
- **Benefits**: High scalability, no complex joins, and low-latency lookups.

High-level Design

The system utilizes a multi-layered approach to ensure performance:

1. **Traffic Management**: Geolocation-based DNS routes users to the nearest data center.
2. **API Gateway**: Handles rate limiting and routes requests to specific microservices.

3. **URL Generation Server:** Standardizes URLs (lowercase, remove ports/params), generates a unique ID via a distributed Snowflake ID generator, and encodes it into Base62.
4. **Redirection Server**: Performs high-speed lookups.
5. **Caching Layer**: Uses a multi-tier cache (Browser, CDN, and Redis) to serve "hot" links at the edge.



Design deep dive

Unique URL creation

To generate the short URL we will take these steps:

Step 1 - Normalize the url

- Convert it to lower case.
- Remove any ports.
- Remove query parameters.
- Normilaze trailing slashes.

For example, these two URLs will map to the same short URL instead of creating a new one for each:

```
https://Example.com
https://example.com
```

Step 2 - Generate a unique ID

Rather than hashing the URL (which introduces collision risk and non-deterministic retry logic), we use a distributed Snowflake-style ID generator. This produces a globally unique 64-bit integer composed of a timestamp, worker ID, and sequence number.

1. Zero collision risk — IDs are unique by construction; no retry loop needed.
2. Deterministic O(1) write latency — no salting or rehashing on conflict.
3. Time-ordered — useful for range queries and debugging.

128 bits is too long for a short URL, so in this step we will take the first 48 bits (6 bytes), convert it to decimal, and then apply Base62.

```
short_code = base62_encode(snowflake_id & 0xFFFFFFFFFFFF)  # lower 48 bits
```

Why Base62?

It uses a-z, A-Z and 0-9, which is URL-friendly. On the other hand, Base64 is avoided because it uses + and / which require URL encoding.

Fast URL Redirection

The redirection service is the performance-cricital component in the system since reads are more than writes. Each time a click happened the service must:

1. Resolve the short url into the long one
2. Redirect the user with minimal latency.

Choosing the right redirection response code:

The read path

1. If there is a 301 (permanent redirect, opt-in only), it is cached in the browser and returned directly. Note: 302 is the default; 301 should only be used when the user has explicitly opted out of analytics.
2. Otherwise, check the CDN. On a cache hit, return the redirect.
3. On a CDN miss, the request is forwarded to the Redirection Server.
4. Check Redis. On a cache hit, return the redirect. The CDN caches it for future requests.
5. On a Redis miss, query the DB. Cache the mapping in Redis with a TTL aligned to the expiration_date, return 302 to the CDN, which also caches it.

Supporting Custom Aliases

Support custom readable URLs instead of auto-generated ones.

If a custom_alias is provided in the API request, it must be validated, checked for uniqueness, and handled atomically to prevent race conditions.

What is a valid custom_alias?

- It is a combination of lowercase letters, uppercase letters, digits, underscores, or hyphens.
- its length between 3 and 50 chars.
- Doesn't contain any reserved words.
- Regex: ^[a-zA-Z0-9_-]{3,50}$

Handling uniqueness and race conditions atomically:

To handle custom aliases reliably and prevent two people from grabbing the same one at the same time, you should follow these steps:

- **Check the format first**: Before talking to the database, make sure the alias follows your rules for length and characters.
- **Use a "Put if Absent" command**: Instead of checking if the alias exists and *then* saving it, send a single command to the database to insert it only if it isn't already there.
- **Handle the error**: If the database tells you the name is already taken (a conflict), simply return an **HTTP 409 Conflict** error to the user.

Support link expiration

Without proper expiration handling, the database could grow endlessly, degrading performance and increasing storage costs. We use two complementary mechanisms.

Backend job

A scheduled process runs weekly to delete expired links from the DB:

```
DELETE FROM db WHERE expiration_date < NOW();
```

This keeps the DB clean, but a link could expire after the last job run. The real-time check handles this gap.

Real-time check

Before returning the long URL, the redirection service checks the expiration_date:

```
if link.expiration_date < current_time:
    return 410 Gone
else:
    redirect_to(link.long_url)    # 302
```

Cache TTL alignment

When caching a mapping in Redis, set the TTL to match the URL's expiration_date rather than a fixed duration. This ensures Redis auto-evicts the key at the right time and eliminates stale cache entries:

```
ttl = expiration_date - current_time
redis.set(short_url, long_url, ex=ttl)
```

Handling High Availability

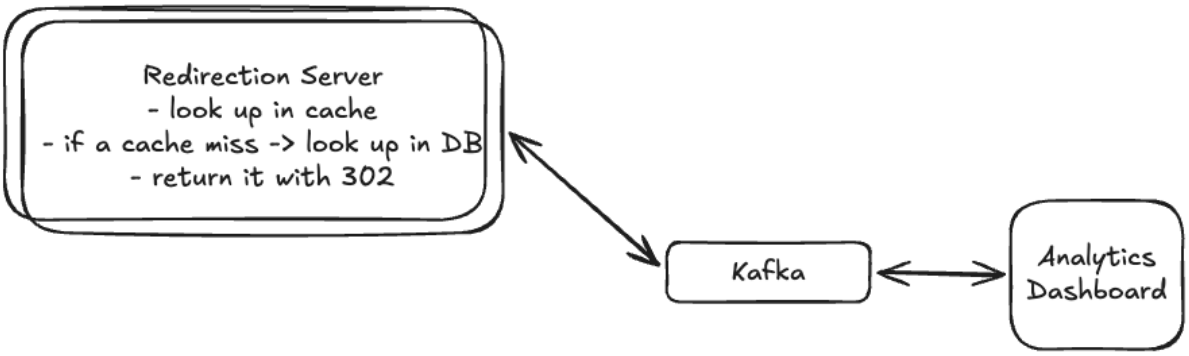
To minimize latency and ensure a seamless redirection flow, the architecture leverages a Multi-Region Deployment strategy:

- **Global Traffic Management:** Geolocation-based DNS routing directs users to the nearest data center, reducing initial connection time (RTT).
- **Active-Passive Write Strategy:** A Primary Write Region handles all URL generation and database writes to ensure strict data consistency.
- **Active-Active Read Strategy:** Redirection is handled by the nearest regional service. Each region maintains its own Read Replicas or a globally replicated NoSQL database (e.g., DynamoDB Global Tables) for local, low-latency lookups.

- **Edge Caching:** Integrating a CDN or Edge Workers ensures high-traffic hot links are served from the network edge, bypassing origin servers entirely.
- **Graceful Degradation & Fault Tolerance:** In the event of a database or regional failure, the system prioritizes availability over consistency. Redirects are served from the Distributed Cache or return stale data rather than surfacing an error to the user.

Click Count

Implementing a real-time click counter introduces significant challenges around performance, data consistency, and high-throughput aggregation.

In a standard flow, the service must both redirect the user and log click metadata. If the click is recorded synchronously, the database write overhead adds measurable latency to the redirection, degrading the user experience.

To maintain low-latency performance, we decouple these processes by making the redirection synchronous and the analytics logging asynchronous. We employ an event-streaming architecture: the Redirection Server generates a click event and pushes it to a Kafka pipeline. A downstream worker process consumes these events to update the Analytics Database without impacting the critical path of the user's redirect.